# GPT-2 from Scratch in C++ and HIP

*A project plan for getting back into C++ through a serious inference engine*

Edin
October 2026

**Contents**


- Why this project
- How to work on it
- When to do it
- Phase 0: Environment and C++ warm-up
- Phase 1: CPU forward pass, validated
- Phase 2: Naive HIP kernels
- Phase 3: Optimization
- Phase 4: int8 quantization and error analysis
- Phase 5: Publishing
- Optional follow-ups
- Resource list


## Why this project

The goal is to rebuild fluency in modern C++ by doing something hard and real, not by working through exercises. You'll write a small but complete inference engine for GPT-2 small (124M parameters). It runs on your Radeon RX 7900 XTX through HIP, it's validated layer by layer against PyTorch, and at the end it's quantized to int8 with a documented error analysis.

Three reasons this is the right project rather than just a fun one:

- **It builds the skills that inference and edge-deployment work hires for.** You'll do memory layout, kernel writing, profiling, and quantization in C++. That's the layer where llama.cpp, ONNX Runtime, MIGraphX and TensorRT live, and where "make the model run reliably on this hardware" problems get solved.
- **It's a verification project at heart.** The final artifact isn't "I wrote a fast kernel." It's "I built an engine, and here is quantified evidence of how far it diverges from the reference, layer by layer, in fp32 and int8." That sits squarely on the Verifiable AI and safety-critical positioning.
- **It's sized right.** GPT-2 small fits trivially in 24 GB, its architecture is fully documented, and the reference implementation is one `pip install` away. Nothing in the project is blocked on hardware or data you don't have.

What you'll end up with: a public repository, a README with benchmark and validation tables, and a write-up you can publish on your own site.

## How to work on it

Read this section twice. Side projects rarely die from difficulty. They die from losing momentum between sessions.

**Always end a session with a breadcrumb.** Before you close the laptop, write one line in a `NOTES.md` saying exactly what to do next ("make `test_layernorm` pass, the epsilon is probably wrong"), or leave a failing test that encodes it. The first ten minutes of an after-work session are the most fragile. A breadcrumb means you start by doing, not by remembering.

**Match tasks to your energy.** Keep two kinds of work on the list:

- *High-energy tasks:* debugging a kernel, designing the tensor abstraction, chasing a numerical mismatch. Save these for weekend blocks or your best evenings.
- *Low-energy tasks:* reading a PMPP chapter, writing a unit test, cleaning up CMake, adding a benchmark table to the README.

On a tired Tuesday, do a low-energy task and still count it as progress.

**Test first, always against a reference.** Every operator you write gets a test that compares it against a known-good output before you move on. First the reference is PyTorch. Once your CPU path is validated, the CPU path becomes the reference for the GPU path. This discipline is the whole verification story, and it also saves enormous debugging time. You'll know immediately which layer broke.

**Commit small and often.** One operator, one test, one commit. If a refactor goes badly, you roll back an hour, not a week.

**Use me as a reviewer, not a ghostwriter.** You're here to get fluent again, so write the code yourself and bring it to me for review. The highest-value questions are "what would a senior C++ developer change here?", "why does this compile error happen?", and "my output diverges from PyTorch after layer 4, help me narrow it down." Keep the plan and your current code in a claude.ai Project so every conversation starts with full context. When you want something built and run directly on your machine, Claude Code in your terminal can work against your actual GPU.

**Check your employment contract before publishing.** Software created as part of your job duties typically belongs to the employer under Swedish copyright law, while personal projects done on your own time and hardware usually don't. Some contracts add broader clauses, though. A five-minute read of the IP section before you push the repo public is cheap insurance. This isn't legal advice, just a known trap.

## When to do it

Yes, after work is realistic, if you plan for it honestly. A sustainable rhythm alongside a full-time job looks like this:

| Slot | Duration | Best for |
|---|---|---|
| Two weekday evenings (e.g. Tue, Thu) | 90–120 min each | Steady progress, tests, reading |
| One weekend block | 3–4 hours | Hard debugging, new phases, optimization |
| **Total** | **~7–10 h/week** | |

Don't plan five evenings a week. You'll do it for two weeks, burn out, and then feel guilty for a month. Two evenings plus a block you protect is better than five you skip.

At that pace the phases below add up to roughly **120–170 hours, or about four to five months**. The optimization phase is open-ended by nature. Set a time budget for it and stop when it's spent, not when the kernel is "done."

| Phase | What | Estimated hours |
|---|---|---|
| 0 | Environment and C++ warm-up | 15–25 |
| 1 | CPU forward pass, validated against PyTorch | 25–35 |
| 2 | Naive HIP kernels | 20–30 |
| 3 | Optimization: matmul, attention, KV cache | 35–50 (time-boxed) |
| 4 | int8 quantization and error analysis | 20–30 |
| 5 | Write-up and publishing | 5–10 |

## Phase 0: Environment and C++ warm-up

The aim here is to get your toolchain solid and your C++ hands warm on low-stakes code before you touch anything numerical.

### Refreshing C++

You're not learning C++ from scratch. You're re-activating it and updating it, since C++ has changed a lot since C++11. The fastest route is a compact overview, then learning by doing, with good references at hand.

- **A Tour of C++, 3rd edition** (Stroustrup, 2022). About 300 pages, written for people who already program. Read it over two or three weeks, in parallel with the warm-up tasks rather than before them. Pay particular attention to the chapters on resource management, move semantics, templates and concepts.
- **learncpp.com**. Free and current. Use it as a lookup when something in the Tour is too terse, not as a course to complete.
- **cppreference.com**. The definitive standard-library reference. Keep it open.
- **CppCon "Back to Basics" talks** on YouTube. Search for the track by name. The talks on move semantics, smart pointers, RAII and templates are each about an hour and are excellent refreshers on a tired evening.
- **Compiler Explorer**. Paste a function and see the generated assembly. You'll use this a lot in Phase 3 to check whether loops vectorize.

> **Hint: the five things rusty C++ developers get wrong.** Raw `new`/`delete` instead of `std::unique_ptr` and RAII. Passing large objects by value. Missing `const`. Not knowing when a move happens versus a copy. Undefined behavior from out-of-bounds indexing that "works" until it doesn't. The sanitizers below catch the last one, and code review catches the rest.

### Toolchain

- **Compiler:** GCC or Clang on CachyOS, targeting C++20. Use `hipcc` (or CMake's HIP language support) for GPU code.
- **Build system:** CMake. **An Introduction to Modern CMake** is free and teaches the target-based style. Avoid tutorials that set global variables like `CMAKE_CXX_FLAGS` everywhere.
- **IDE:** **CLion is free for non-commercial use** and will feel familiar coming from PyCharm. It understands CMake projects natively.
- **Testing:** **GoogleTest**, pulled in with CMake's `FetchContent`. Every operator gets a test.
- **Sanitizers:** build your debug configuration with `-fsanitize=address,undefined`. This catches out-of-bounds access, use-after-free and signed-overflow bugs the moment they happen, instead of three layers later as garbage numbers. Treat any sanitizer report as a stop-everything bug.
- **Python reference environment:** a separate virtual environment with PyTorch (the CPU build is fine for reference dumps), `transformers`, `safetensors`, `tiktoken` and `numpy`.
- **ROCm:** you already run llama.cpp on ROCm, so the driver side works. For HIP development you need the HIP SDK (`hipcc`, hipBLAS for the performance ceiling later) installed and matching versions.

> **Hint: pin the GPU.** ROCm may also see your 7600X's integrated GPU. Set `HIP_VISIBLE_DEVICES=0` (and check with `rocminfo` that device 0 is the 7900 XTX) so your engine never accidentally lands on the iGPU. Running the display from the iGPU while you develop is a good habit: long-running debug kernels won't trigger display watchdog timeouts.

### Warm-up task 1: a safetensors loader

GPT-2 weights from Hugging Face (`openai-community/gpt2`) come as a `model.safetensors` file. The format is beautifully simple, which makes it the perfect first exercise:

1. The first 8 bytes are an unsigned 64-bit little-endian integer `N`.
2. The next `N` bytes are a JSON object mapping tensor names to `{dtype, shape, data_offsets: [begin, end]}`, plus an optional `__metadata__` entry.

3. Everything after that is raw tensor data. The offsets are relative to the start of this data section, not the start of the file.

Write a class that opens the file, parses the header (use nlohmann/json rather than writing a JSON parser), and gives you named, typed, shaped views into the data.

> **Hint:** This exercise touches file I/O, binary parsing, ownership, and lifetimes. Do it twice. First, read the whole file into a `std::vector<std::byte>` and hand out views. Then do it properly with `mmap`, wrapped in an RAII class whose destructor unmaps. Ask yourself who owns the memory and what happens if the loader is destroyed while a view still exists. That question is the heart of modern C++.

> **Hint:** Print the header first, before writing any model code. Look at the tensor names and shapes with your own eyes. You'll see names like `h.0.attn.c_attn.weight` and `wte.weight`. Knowing the real names and shapes up front prevents a lot of confusion later.

### Warm-up task 2: a minimal Tensor

Write the smallest `Tensor` type that's useful: a shape, strides, a dtype (start with `float32` only), and storage. Give it element access, a pretty-printer for debugging, and a function to load reference arrays dumped from Python (raw binary plus a small shape header is enough; you don't need `.npy` parsing).

> **Hint:** Resist the urge to build a general framework. You need maybe ten operations for GPT-2. A small, honest tensor type you understand completely beats a clever one that fights you. Separate *storage* (who owns the bytes) from *view* (shape, strides, offset), and you'll thank yourself in Phase 2 when storage can live on the GPU.

## Phase 1: CPU forward pass, validated

Now implement GPT-2 inference on the CPU in straightforward C++, with no optimization at all. The output of this phase is a correct engine and a validation harness, which together become the reference for everything that follows.

### Learn the architecture

- **GPT in 60 Lines of NumPy** (Jay Mody). The cleanest conceptual blueprint of the GPT-2 forward pass. Keep it open beside your editor; your C++ will map almost line for line onto it.
- **llm.c** (Karpathy), specifically `train_gpt2.c`. A complete GPT-2 in plain C, CPU only, in one readable file. It also ships a PyTorch reference script that dumps activations to compare against, which is exactly the shape of your validation harness. Read it to understand structure, then close it and write your own version. Don't copy.
- **Karpathy's "Let's build GPT" and "Let's reproduce GPT-2" videos** on YouTube, if you want the narrative version.

### The model, in brief

GPT-2 small has 12 transformer blocks, 12 attention heads, a hidden size of 768, a context length of 1024, and a vocabulary of 50,257 tokens. The forward pass:

1. Token embedding (`wte`) plus position embedding (`wpe`).
2. Twelve blocks, each: LayerNorm → causal self-attention → residual add → LayerNorm → MLP (768 → 3072 → GELU → 768) → residual add.
3. A final LayerNorm (`ln_f`).
4. Logits: multiply by the transposed token-embedding matrix (the weights are tied, so there's no separate output matrix).

### The traps (read before you start)

These are the places where nearly everyone's first implementation diverges from PyTorch:

- **Weight layout.** Hugging Face's GPT-2 uses a `Conv1D` module rather than `nn.Linear`, so weights are stored as `[in_features, out_features]`. The computation is `y = x · W + b` with no transpose. If your outputs are garbage from layer 0, check this first.
- **The fused QKV projection.** `c_attn.weight` is `[768, 2304]`: the query, key and value projections concatenated along the last dimension. Split it into three 768-wide pieces, then split each into 12 heads of 64.
- **GELU variant.** GPT-2 uses the tanh approximation, `0.5·x·(1 + tanh(√(2/π)·(x + 0.044715·x³)))`, not the exact erf form. Using the wrong one gives small errors that grow with depth, which is very confusing to debug.
- **LayerNorm epsilon** is `1e-5`.
- **Attention scaling** is `1/√64`, with a causal mask (each position attends only to itself and earlier positions).
- **Numerically stable softmax:** subtract the row maximum before exponentiating.
- **Hugging Face's `hidden_states`:** if you dump reference activations with `output_hidden_states=True`, the *last* entry already has `ln_f` applied, unlike the others. Use forward hooks for finer, unambiguous dumps.

### The validation harness

Write a Python script that runs the reference model on a fixed prompt and dumps intermediate tensors: embeddings, each block's attention output and MLP output, final hidden state, and logits. Run the reference **on the CPU in float32** so the reference itself is deterministic. Tokenize with tiktoken (the `gpt2` encoding) and pass the token IDs to both implementations. Writing your own BPE tokenizer is a detour; skip it for now.

Your C++ tests then load each dump and compare against your output, reporting maximum absolute error and maximum relative error per tensor.

> **Hint: what to expect.** You won't match bitwise, because floating-point addition isn't associative and your summation order differs from PyTorch's. Expect maximum absolute errors around `1e-5` to `1e-4` in float32, growing gradually with depth. A sudden jump at a specific layer is a bug. A smooth, slow growth is arithmetic. Learning to tell those apart is the core verification skill.

**Phase 1 is done when:** every intermediate tensor matches within tolerance, and greedy generation from a prompt produces the same token sequence as PyTorch for at least 50 tokens.

> **Hint:** Save the per-layer error numbers from this phase. They're the fp32 baseline your int8 results get compared against in Phase 4, and they're the first table in your final write-up.

## Phase 2: Naive HIP kernels

Port each operator to the GPU in the simplest possible way. The goal is correctness on the GPU and a working understanding of the HIP programming model. Performance comes in Phase 3.

### Resources

- **HIP documentation**, particularly the programming guide sections on the programming model, memory management and kernel launch.
- **rocm-examples**. AMD's official sample repository. Find the simplest example with a CMake setup and copy its structure for your build.
- **Programming Massively Parallel Processors, 5th edition** (Hwu, Kirk, El Hajj, 2026). Your main GPU textbook for Phases 2 and 3. It's written in CUDA, but HIP is designed to mirror CUDA's API, so the code translates almost directly. Read the opening chapters on the execution model, memory hierarchy and performance considerations now.

### The work

1. **An error-checking macro.** Every HIP call returns a `hipError_t`. Wrap every call in a macro that checks it and reports file and line. Do this before anything else; unchecked errors are the number one source of "it just returns zeros."
2. **An RAII device buffer.** A class that calls `hipMalloc` in its constructor and `hipFree` in its destructor, is movable but not copyable, and has explicit `upload` and `download` methods. This is your Phase 0 ownership thinking paying off.
3. **One kernel per operator:** embedding lookup, LayerNorm, matmul, attention (start with a separate softmax kernel), GELU, residual add. Write them naively: one thread per output element is fine.
4. **Test each kernel against your CPU implementation**, which is now your reference. Same tolerance discipline as before.

> **Hint: debugging.** During development, synchronize and check for errors after every kernel launch. AMD's `AMD_SERIALIZE_KERNEL` environment variable makes kernel launches synchronous, which makes errors show up at the launch that caused them. Turn it off once things work.

> **Hint: wavefront size.** On RDNA3 (your card, `gfx1100`), HIP compiles with 32-wide wavefronts by default, the same width as an NVIDIA warp, so CUDA-oriented material maps cleanly. Still, never hard-code 32. Use the built-in `warpSize`. AMD's datacenter GPUs use 64, and portable code is part of the point.

> **Hint: timing.** Measure kernels with `hipEvent` timers, not CPU wall clock, and always warm up with a few untimed runs first. The first launch includes one-time setup costs.

**Phase 2 is done when** the full forward pass runs on the GPU and matches your CPU engine within tolerance, and generation produces the same tokens.

## Phase 3: Optimization

This is the fun part and the dangerous part: it's open-ended. **Set a time budget (around 40 hours) and stop when it's spent.** Write down what you'd do next instead of doing it.

### Measure first

Before optimizing anything, profile. Use **rocprofv3** (from the ROCm profiler SDK) to see which kernels dominate total time. For GPT-2 inference, the matrix multiplications will almost certainly dominate, with attention second.

Learn **roofline thinking**: for each kernel, compute how many floating-point operations it does per byte of memory it moves, and compare against your card's compute peak and memory bandwidth (the 7900 XTX has about 960 GB/s). This tells you whether a kernel is limited by memory or by compute, and therefore which optimizations can possibly help. PMPP covers this well.

> **Hint:** Use hipBLAS/rocBLAS matrix multiplication as your performance ceiling. Benchmark it on your exact matrix shapes. Your goal isn't to beat it. Getting to 50–80% of it with your own kernel is a genuinely strong result, and understanding *why* the remaining gap exists is the real learning.

### The matmul ladder

**How to Optimize a CUDA Matmul Kernel for cuBLAS-like Performance** (Simon Boehm) is the standard step-by-step worklog. It goes from a naive kernel to near-library performance, measuring each step. Work through it, porting each step to HIP and measuring on your card:

1. Naive kernel (you already have it).
2. Coalesced global memory access.
3. Shared-memory tiling.
4. Thread-level register blocking (each thread computes several outputs).

5. Vectorized loads.
6. Tuning tile sizes for your hardware.

Keep a table of throughput after each step. That table goes in your write-up.

> **Hint:** The step numbers from an NVIDIA blog won't transfer exactly to RDNA3. Different cache sizes, register file and scheduling mean different optimal tile sizes. Explaining *why* your best configuration differs from the blog's is more impressive than matching its numbers.

### Attention and generation

- **Fuse softmax** into the attention computation instead of writing the full score matrix to memory and reading it back.
- **Read about online softmax** (Milakov & Gimelshein, 2018) and **FlashAttention** (Dao et al., 2022). You don't need to implement full FlashAttention. Understanding the tiling-plus-online-softmax idea and applying a simple version is enough.
- **Add a KV cache** for generation. Without one, every new token recomputes attention over the whole sequence. With one, generating each token is dramatically cheaper. This is the single biggest practical speedup for text generation, and a nice correctness test: cached and uncached generation must produce identical tokens.

### Stretch: matrix cores

RDNA3 has WMMA instructions for fast half-precision matrix math. AMD's **How to accelerate AI applications on RDNA 3 using WMMA** walks through using them on your card specifically. Treat this as optional. It's worth doing only if you've got budget left and the fp32 path is solid.

## Phase 4: int8 quantization and error analysis

This phase turns a performance project into a verification project, and it's the part most aligned with your positioning.

### The work

1. **Symmetric per-output-channel weight quantization.** For each output channel of each weight matrix, compute a scale (`max|w| / 127`) and store weights as int8 plus one float scale per channel. Quantize offline in your loader.
2. **Dequantize inside the matmul kernel**, keeping activations in float32 (often called W8A32). This is the simplest correct scheme and isolates the error introduced by weight quantization alone.
3. **Optional extension:** quantize activations too (W8A8) and compare. This is where error grows non-trivially and where outlier activations start to matter.

### The analysis

- **Per-layer error** against the fp32 reference: the same harness as Phase 1, now plotted alongside your fp32 baseline.
- **Token agreement:** over a set of prompts, how often does greedy decoding pick the same next token as fp32?
- **Perplexity** on a small text sample (a slice of WikiText-2 is standard), evaluated by your own engine in fp32 and int8. This turns "the numbers are a bit different" into a single interpretable quality metric.
- **Memory and speed:** weight memory before and after, and tokens per second.

> **Hint:** Write the analysis up as if it were a short validation report for a customer: what you tested, how, what the tolerances were, what the results were, and where the method's limits are. That framing is the point. It shows you can reason about whether an optimized model is still *trustworthy*, which is a much rarer skill than making it fast.

## Phase 5: Publishing

- **README:** what the engine does, how to build and run it, and the three key tables (fp32 validation errors, the matmul optimization ladder, int8 quality and performance).
- **A write-up** on your site covering the journey and the validation results. Lead with what you verified and how, not just how fast it is.
- **Be honest about limits.** Single GPU, GPT-2 small only, the specific quantization scheme chosen. Clear statements of scope read as engineering maturity.

## Optional follow-ups

Once the main project is done, any of these extends it naturally:

- **Port the CPU reference to Rust** and run it against the same validation harness. "The same model, two languages, one verification suite" is a strong cross-language story. For Rust GPU work on AMD, Burn and its CubeCL kernel language have a ROCm backend. For the safety-critical angle, note that Ferrocene is a Rust compiler qualified for ISO 26262 and IEC 61508.
- **Scale up to GPT-2 medium (355M)** and see what breaks: memory planning, tile sizes, error growth over 24 layers.
- **Compare against llama.cpp** running the same model on the same card, and explain the performance gap by reading its HIP backend.
- **Study framework design** through **CMU 10-414/714 Deep Learning Systems**, where you build a small framework with autograd and CPU/GPU backends. Writing a HIP backend for it would be a natural continuation.
- **Go deeper on efficiency** with **MIT 6.5940 TinyML and Efficient Deep Learning Computing** (Song Han): quantization, pruning and distillation done properly, with lectures freely available.
- **Sharpen CPU performance skills** with **Performance Analysis and Tuning on Modern CPUs** (Bakhvalov), free online, and use it to optimize your CPU path with SIMD and cache-aware loops.

## Resource list

**C++**

- A Tour of C++, 3rd ed.
- learncpp.com
- cppreference.com
- Compiler Explorer
- An Introduction to Modern CMake
- GoogleTest
- nlohmann/json
- CLion (free for non-commercial use)
- CppCon "Back to Basics" track (YouTube)

**The model**

- GPT in 60 Lines of NumPy
- llm.c
- GPT-2 on Hugging Face
- safetensors format
- tiktoken

**GPU programming**

- Programming Massively Parallel Processors, 5th ed.
- HIP documentation

- rocm-examples
- How to Optimize a CUDA Matmul Kernel (Boehm)
- WMMA on RDNA 3 (GPUOpen)
- GPU MODE lectures
- Online softmax (Milakov & Gimelshein)
- FlashAttention (Dao et al.)

**Going further**

- CMU 10-414/714 Deep Learning Systems
- MIT 6.5940 Efficient Deep Learning Computing
- Performance Analysis and Tuning on Modern CPUs
- Burn (Rust)
- Ferrocene